GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

从智能走向自主：开放环境下具身智能体的稳态AI压力门控控制方法

From Intelligence to Autonomy: The Case for Homeostatic AI Stress-Gated Control for Embodied Agents in Open-Ended Environments



Bartlomiej Szejny

AI Poland董事会成员

Board Member, AI Poland



GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

DHttp: From partial connectivity to omniconnectivity

Presenter: Daryl Wong
Co-Founder of Genmeta Tech



CONTENTS

Part 01 Background & Current Web Landscape

Part 02 The Core Structural Problem of HTTP

Part 03 The DHttp Solution & Protocol Architecture

Part 04 Practical Usage, Tooling & The Next Web Era



01

Background & Current Web Landscape

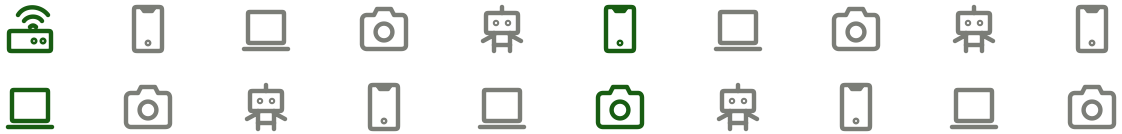


Is the Current Internet Truly "Interconnected"?

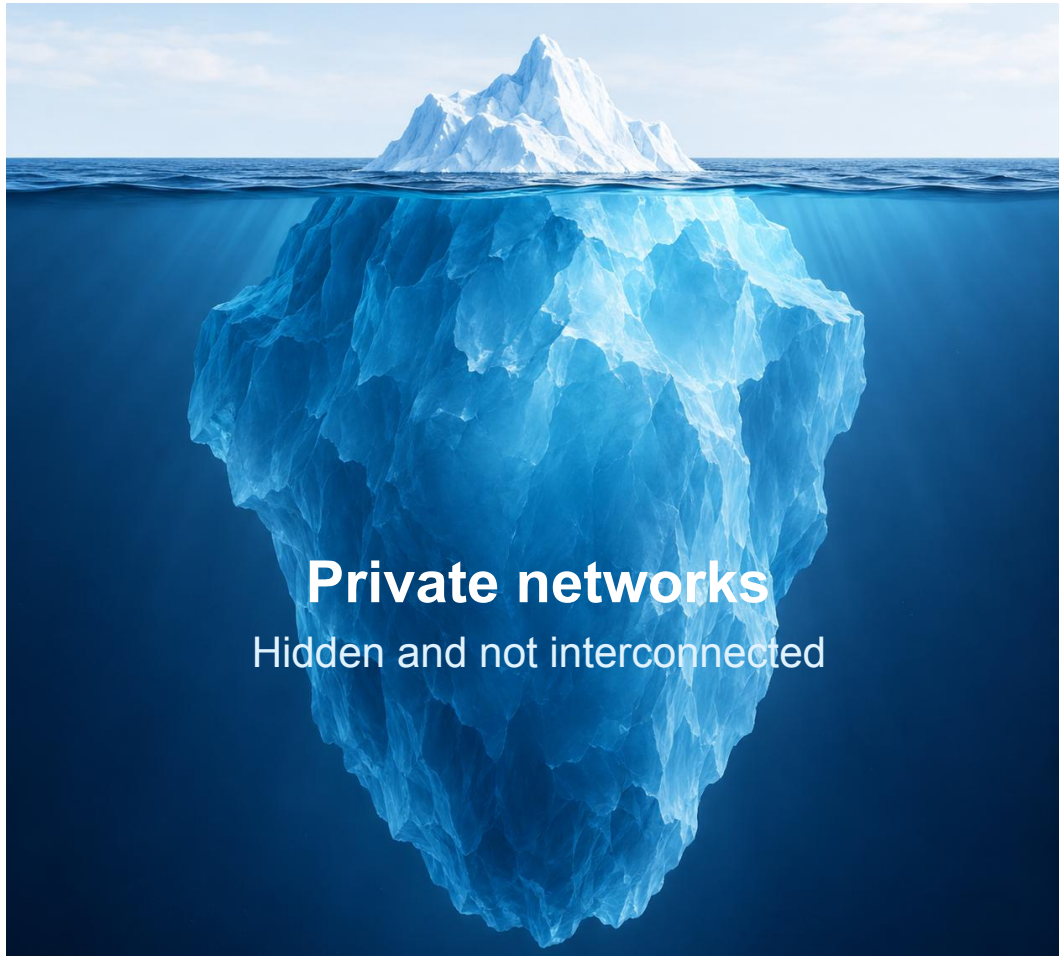
≈4B IPv4 addresses

vs

≈40B connected devices

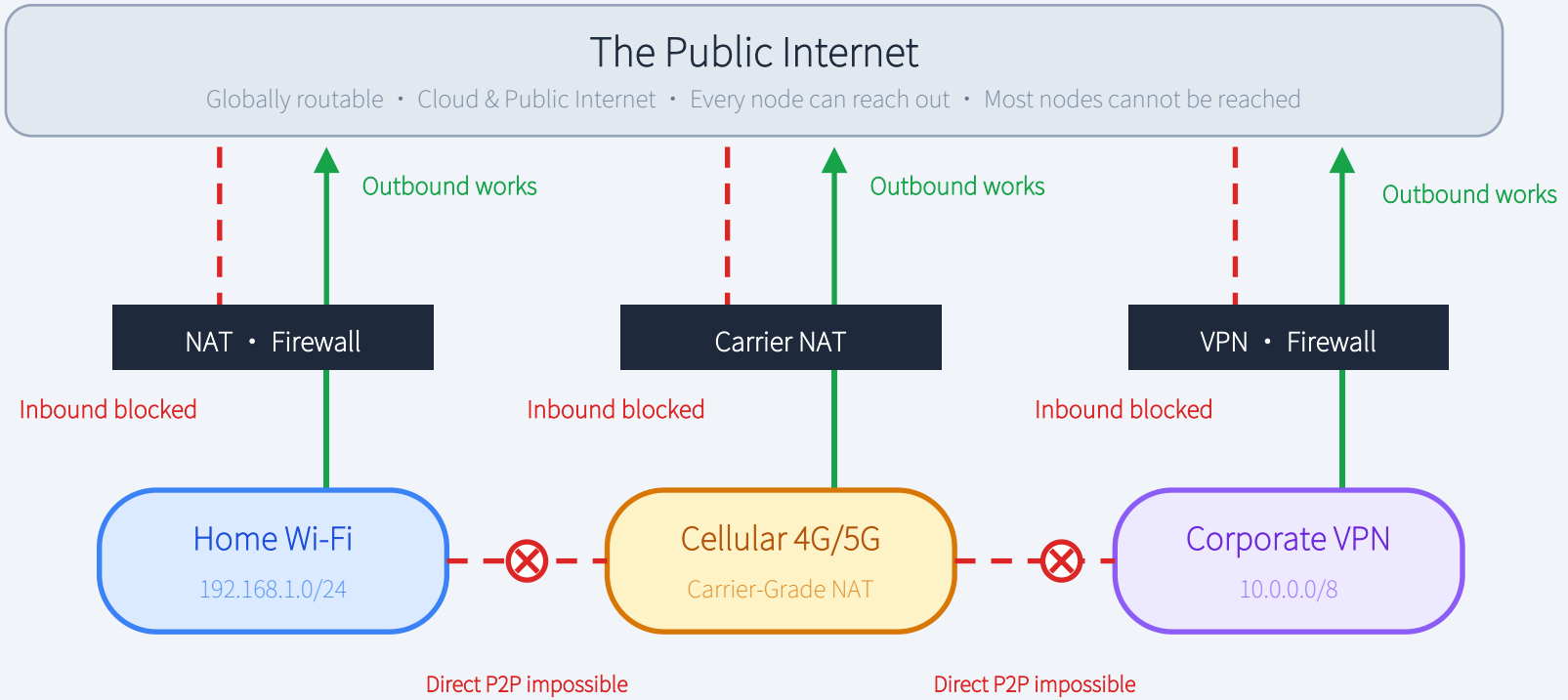


About 1 in 10 devices could over hold a public address



Invisible private networks are a part of the Internet we cannot ignore.

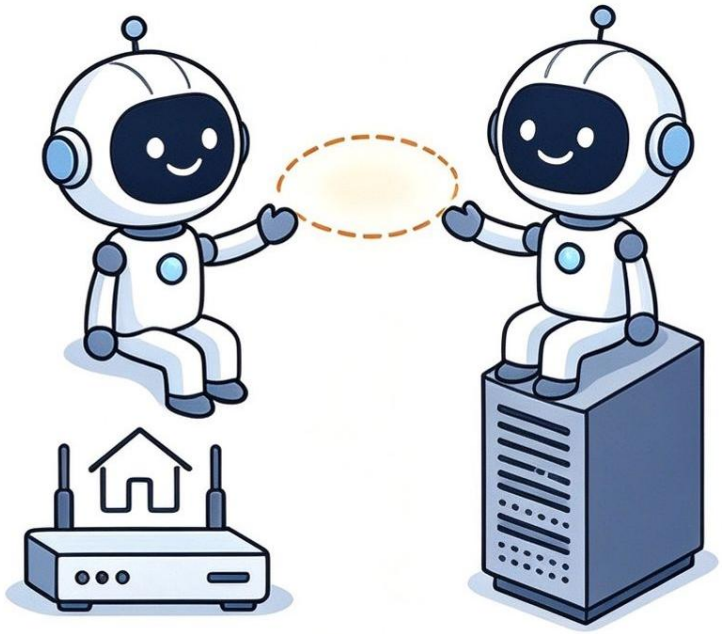
Partially Interconnected: The Private Network Trap



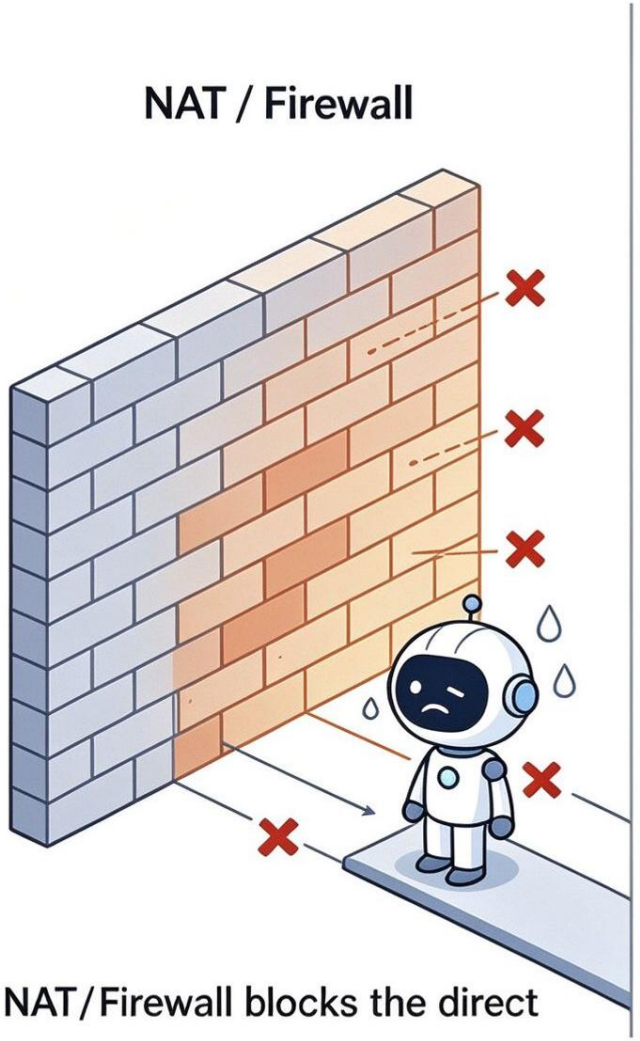
cross-network	connectivity
public → public	Yes
private → public	Yes
public → private	No
private → private	No

This is the root cause: for a private-network node, **"connected to the internet"** is not the whole of **"interconnected."** It lacks the ability to be **"connected from the internet"** — and that requires the very first handshake packet to arrive, which demands **reachability from the internet**, i.e., the node must be **globally routable**.

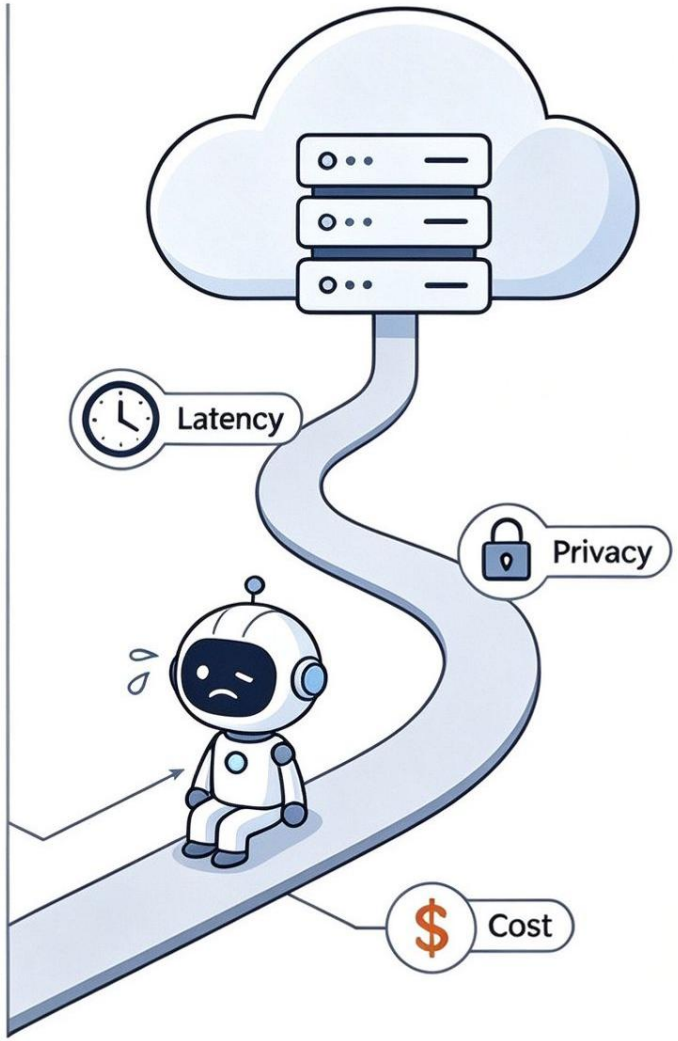
Why need Omniconnectivity?



Two agents on separate edge networks reach out for a direct connection



NAT/Firewall blocks the direct path, agents are stuck



Forced through cloud relays – latency, privacy, cost

GOSIM

GOSIM Shenzhen 2026

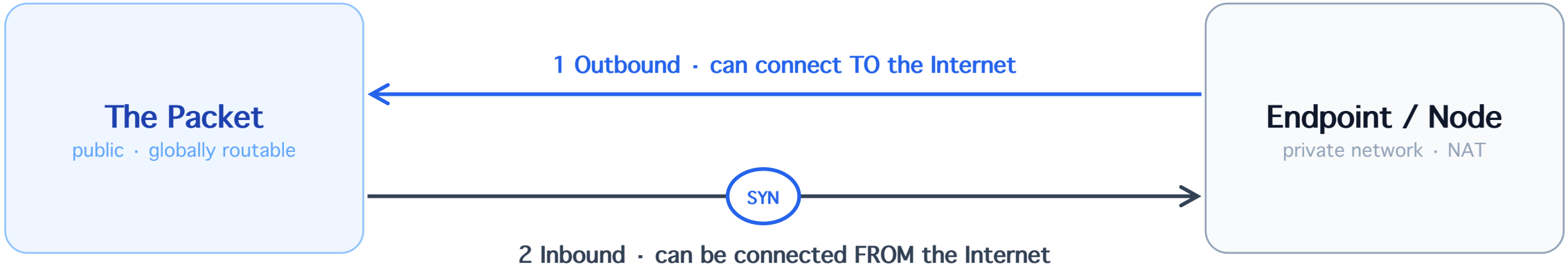
02

Problem Analysis & Deconstructing HTTP



What does true interconnection mean for a node?

True interconnection for one endpoint is bidirectional — not merely reaching out.



3 The inbound **SYN handshake packet** must actually reach the endpoint.

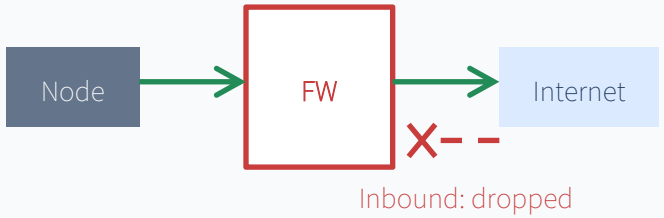
4 That requires the endpoint to be **globally routable** — but in a private network, it **isn't**.

Why are Private-Network Nodes Not Globally Routable?

Three barriers to unsolicited inbound reachability

01 Stateful Firewall

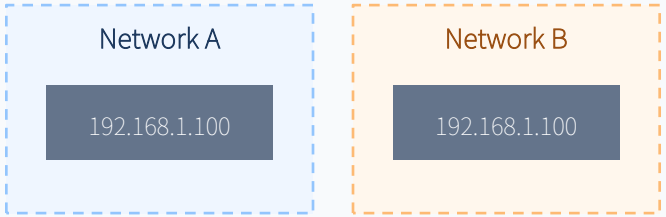
Return traffic only



Only return packets from connections the node itself initiated are allowed through.
Unsolicited inbound traffic is dropped at the firewall.

02 Private IP Overlap

Same address, many networks



Whose IP is 192.168.1.100?

No reply / wrong reply

192.168.1.100 exists in many independent networks.
Routing by IP alone is ambiguous — it does not identify a unique node.

03 NAT / NAPT

Mapping hides the real node



Private addr → Public addr:port

External peers see only the public address

NAT maps private IPs to a public IP, breaking end-to-end routability.
Only Full-Cone NAT allows unsolicited inbound; other types do not.

Together: private nodes can initiate outbound, but cannot be reached directly by any node on the global Internet.

What it takes to be reached: a listening socket + a globally routable address for the first handshake packet

Who actually has both: cloud hosts with static public IPs — full stop

Who doesn't: anything behind NAT, a firewall, or 5G — the handshake packet never even arrives

The tax: cloud relay fees, extra hop latency, and private data streaming through a server that didn't need to see it



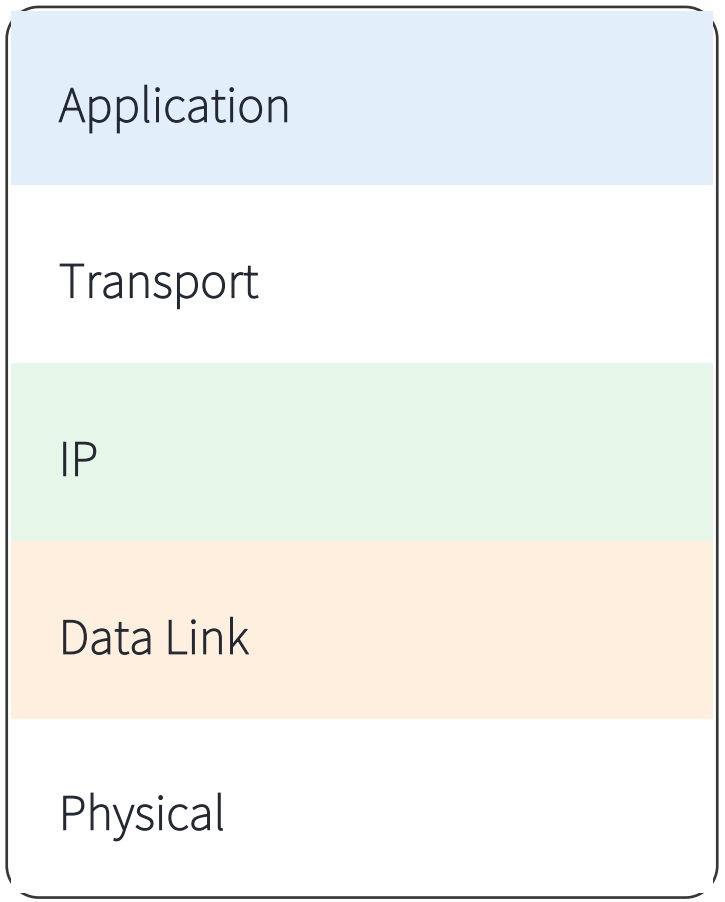
Network Socket Primitives

PRIMITIVE	CALLER	CALLEE	WHAT IT REQUIRES
socket.listen()	A server only	—	Must own a static, publicly routable IP address.
socket.connect(name)	Any endpoint	Server name only — the endpoint being dialed	The target must already be routable on the global internet.
getaddrinfo(name)	Any endpoint	Server name only — the endpoint being resolved	The name must resolve to a globally routable A record — which only a public server can hold. A node inside a private network cannot publish one.



The DNS protocol **serves only the server side** — it has no primitive that names a client.

How can devices in private networks regain global routability like cloud servers?



Name / Address	Every Device?	
Domain Name example.com	No Longer	Belongs to servers
IP Address 192.168.1.8	Every Device	Every device has one
MAC Address A4:C3:F0:9B:12:7D	Every Device	Every NIC has one

GOSIM

GOSIM Shenzhen 2026

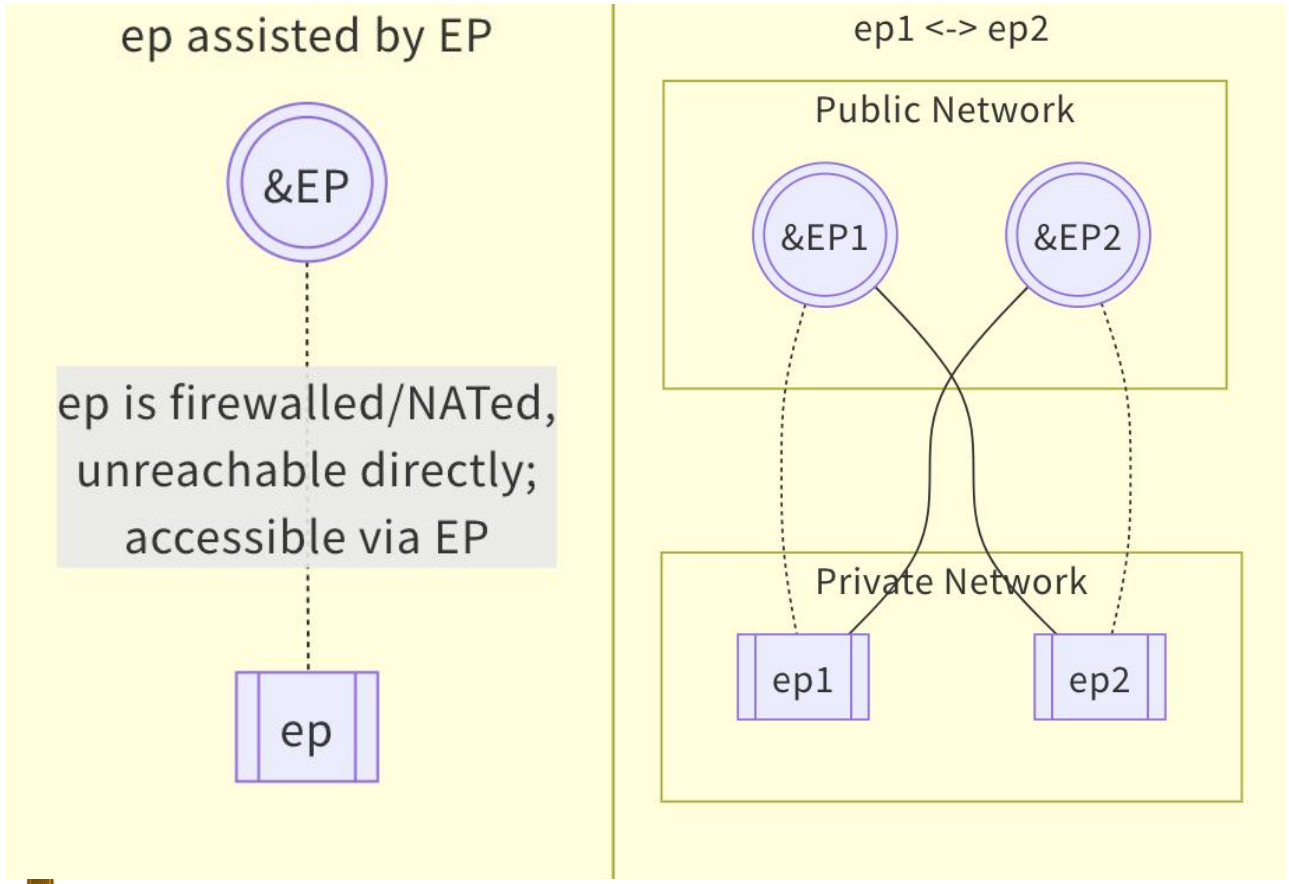
03

The DHttp Solution & Protocol Engineering



How to make private endpoint global routable?

Circles (&EP) = Public IP Endpoints | Rectangles (ep) = Internal Endpoints behind NATs



- Dual-Identity Pair:

  &EP acts as the public proxy/delegate for ep.

  Talking to &EP is functionally equivalent to talking to ep.

  Bidirectional Relay Channel:

  Outbound: ep1 → &EP2 → ep2

  Inbound: ep2 → &EP1 → ep1

```
pub enum EndpointAddr {
    Direct {.              // for EP with Public IP
        addr: SocketAddr,  // which is EP's
    },
    Mediate {.             // for ep in private net
        agent: SocketAddr, // which is &EP's
        outer: SocketAddr, // which is ep's
    },
}
```

DQuic: Quic + Using Quic to Traversal NATs

Universal Domain Names: Resolve to EndpointAddr

Traditional DNS

~~A / AAAA~~

IP addresses only
Geared toward fixed servers

DDns extension

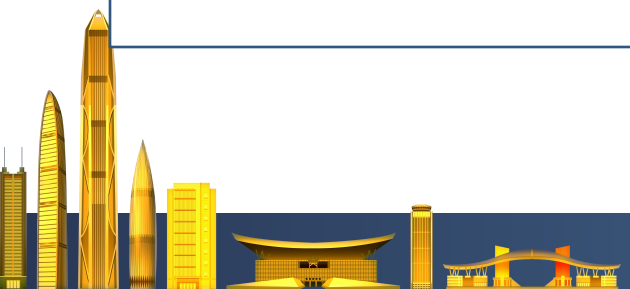
E Record

EndpointAddr: IP + port
+ delegate endpoint + signature

```
genmeta nslookup bob.lee~

# Name      bob.lee~
# Type      E / Endpoint
# Address   1.12.74.4:20004
# Via       45.248.109.11:5072
# Flags     IPv4 · Signed · IDX(2)
```

rtype = 266	flags	rdata



Decentralized Http

DHttp

HTTP/3 + symmetric streams

name~ · mTLS · endpoint equality

DDns

E-Record (RTYPE 266)

self-published · signed

DQuic

QUIC + NAT traversal + multipath

STUN/Relay/Signal, native



1 Resolve the name

Alice.Smith → DDns

lookup "Bob.Lee"

E-Record, RTYPE 266



2 Establish a secure connection

Alice.Smith ↔ DQuic

Connect to Bob.Lee's EndpointAddr
Mutual TLS handshake
Direct, encrypted P2P stream



3 Send the request

Alice.Smith ↔ DHttp

Request to Bob.Lee

What is a name?

More than just a string of digits or characters!

01

Unique Identifier

02

Verifiable

03

Callable

04

Signable

Traditional Identifiers

The digital world has produced many kinds of identities:

Type	Unique	Verif.	Call.	Sign.	Read.	Typical use
IP Address	✗	✗	△	✗	✗	Nginx logs
National ID	✓	✓	✗	✓	✗	Present to prove
Username	?	✗	△	✗	?	Third-party apps
Password	–	✓	–	–	✗	Long-term credential
Access Token	?	✓	–	–	✗	Refreshable
OTP Code	–	✓	–	–	✗	One-time, short-lived
Biometrics	–	✓	–	–	✗	Long-term factor
OIDC	✓	✓	△	✗	?	Cross-app login
Web3 Wallet	✓	✓	△	✓	✗	Blockchain finance
DID	✓	✓	△	✓	✗	Decentralized ID

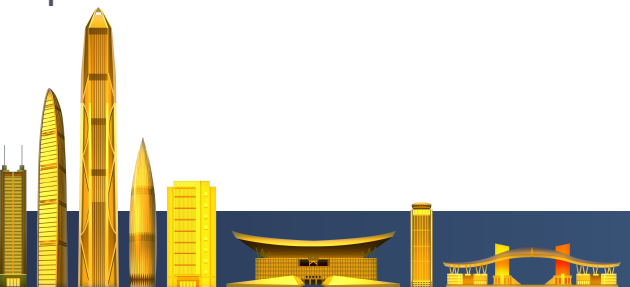
General Identity = Name + Key Pair + Certificate

Use Case

When a name can be called over HTTP

```
https://bob.lee~/exposed/ability
```

Behind a name lies no longer just a web page, but an endpoint exposing its capabilities: profiles, files, chat, agents, code, vehicles, home, and health data. Open interconnection does not mean running naked — every capability is guarded by permissions.



VPN vs. DHttp — Why Tunneling Isn't Enough

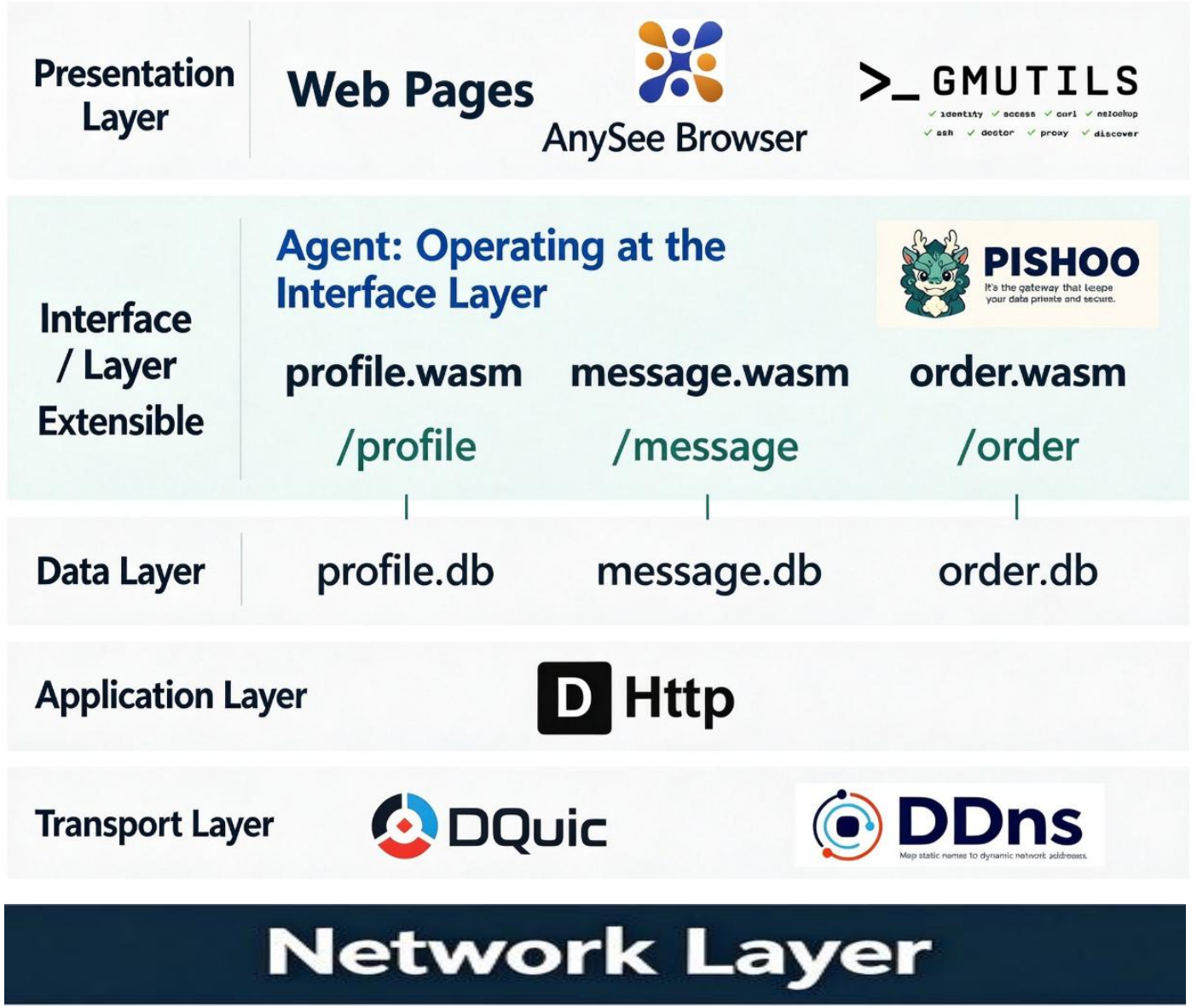
	VPN	DHttp
Reach	Within a single virtual private network	Unrestricted — works across networks
Onboarding	Connect first, then join the virtual network	Works as soon as you are online
Layer	Network layer; carries many upper-layer protocols	Application layer; built around HTTP
Identity	IP address — spoofable	Name + keypair + certificate; mutual TLS (mTLS)
Granularity	(protocol, IP, port) — authorizes incoming IPs or devices	(name, API, Method) — authorizes incoming names
Access control	iptables, ufw, or the VPN's own ACL	genmeta-access

04

Practical Usage, Tooling & Future Paradigm Shift



Developer Utility Tools & Browser/Server Integration



How to use the SDK?

Rust examples with credentialed endpoints

As Client

```
let (me, peer, connection) = endpoint
    .connect("localhost:4433".into()).await?;
let (stream_id, (mut reader, mut writer)) = connection
    .open_bi_stream().await?
    .ok_or("stream IDs exhausted")?;

let message = b"hello, dquic!";
writer.write_all(message).await?;
writer.shutdown().await?;
let mut reply = Vec::new();
reader.read_to_end(&mut reply).await?;
assert_eq!(reply, message);
println!("Echo: {}", String::from_utf8(reply)?);
```

As Server

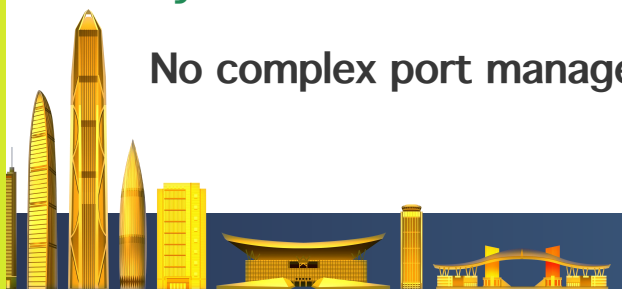
```
endpoint.listen(Loopback, |(peer, me, connection)| {
    tokio::spawn(async move {
        let (stream_id, (mut reader, mut writer)) =
            connection.accept_bi_stream().await?;
        let mut message = Vec::new();
        reader.read_to_end(&mut message).await?;
        writer.write_all(&message).await?;
        writer.shutdown().await?;
        Ok::<(), Box<dyn std::error::Error
            + Send + Sync>>(())
    });
})?;
tokio::signal::ctrl_c().await?;
```

Symmetric client and server APIs

No complex port management

Mutual authentication built in

Even simpler than popular HTTP libraries



Demo: Social App



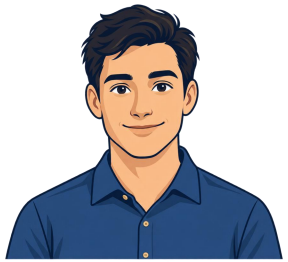
alice.smith

Alice's APIs

```
/std/contact
/std/profile/avatar
/std/profile/nickname
/std/message
```

Bob's APIs

```
/std/contact
/std/profile/avatar
/std/profile/nickname
/std/message
```



bob.lee

Endpoint("alice.smith").post("https://bob.lee~/std/contact")

```
{
  "name": "alice.smith",
  "subject_id": "a8de47…",
  "class": "human",  // not only human, but alse agents/robots/services
  "description": "I am your best friend",
  "request": {
    "/std/profile/avatar": ["GET"],
    "/std/profile/nickname": ["GET"],
    "/std/message": ["POST"]
  },
  "offers": {
    "/std/profile/avatar": ["GET"],
    "/std/profile/nickname": ["GET"],
    "/std/message": ["POST"]
  }
}
```

Adding a friend means negotiating access permissions with each other.

Demo: Agent Cooperation

01 输入任务

POST /task

Host: bob.lee~

任务内容：……

结果返回，继续处理



javis.bob.lee PKI / mTLS

调用别人 → https://alice.smith~/api

被别人调用 ← https://javis.bob.lee~/api

数据聚合	业务逻辑	API 交互
profile.db	profile.wasm	←/profile
message.db	message.wasm	←/message
order.db	order.wasm	←/order

访问权限

access POST /message allow alice.smith~

同步/读数据 / API / 功能

智能体本体

02 查询联系人

联系人及其已授权的 API

alice.smith~ 好朋友

GET /profile/avatar
GET /profile/nickname
POST /messages

walle.robot~ 扫地机器人

POST /task/clean
GET /task/status

trip.com 旅行服务商

POST /flight/book
POST /hotel/book



03 推理，循环发起调用

POST /task/clean

Host: walle.robot

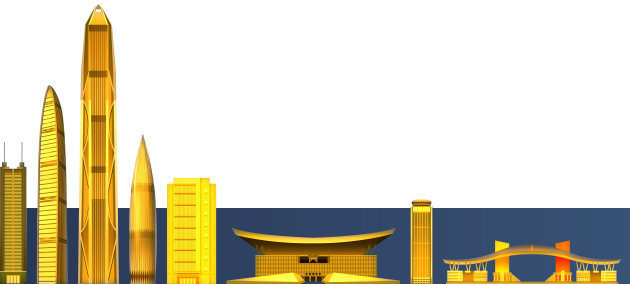
Authorization: Sectary issuer="bob.lee" …

Join the Open Source Movement & Get Started

Rebuilding the Internet into a true two-way street—one name, one keypair, one HTTP request at a time.

Repo: https://github.com/genmeta
Docs: docs.dhttp.net
Core Repositories: dhttp, Pishoo, DQuic
Email: contact@genmeta.net

Quick Start:



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

开源机器人

Open Source Robotics

